① Inception

Introducing React

Q) What is React? Why is it known as "React"?

→ React is a **JavaScript library** . The name "React" was chosen because it was designed to let developers react to changes in the state and data of an application, and update the UI in an efficient and declarative way.

Q) What is a Library?

→ A collection of pre-written code that we can plug into our app and reuse to do certain tasks, which means faster development. Example: React, jQuery.

Q) What is a Framework?

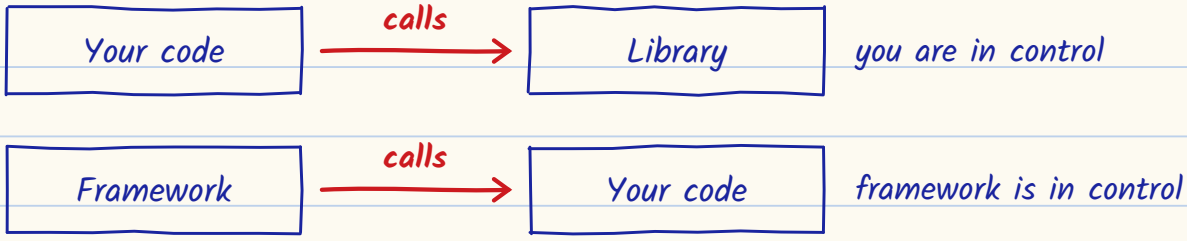
→ It provides a basic foundation / structure for a website or app. Example: Angular.

Q) Similarity between library and framework?

→ Both are code written by third parties to solve common problems and save our time.

Q) Difference between library and framework?

→ The key difference is **Inversion of Control** .



Library: we decide when to call it. Framework: it decides when to call our code.

React is a barebone, minimal library - it can work on a small portion of our app. A framework loads up with everything and gives a fully fledged app.

Emmet:

Emmet is a toolkit for web developers (built into VS Code). We type shortcuts and they expand into full boilerplate code for HTML and CSS.

Try: type div#root>h1{Hello World!} and press Tab.

Hello World - using only HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport"
      content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>
  <body>
    <div id="root">
      <h1>Hello World! using only HTML</h1>
    </div>
  </body>
</html>
```

Hello World - using only JavaScript

Same result, but we create the element with DOM operations.

```
<body>
  <div id="root"></div>
</body>
<script>
  const heading = document.createElement("h1");
  heading.innerHTML = "Hello World! From JavaScript";

  const divElement = document.getElementById("root");

  divElement.appendChild(heading);
</script>
```

Hello World - using only React

Our app doesn't know about React, neither does the browser. So to get React's superpowers we add **CDN** links (Content Delivery Network = servers spread across locations that deliver files faster and avoid too much traffic on the main server).

```
<body>
  <div id="root"></div>

  <script
    crossorigin
    src="https://unpkg.com/react@18/umd/react.development.js"
  ></script>
  <!-- react.development.js has the core React code (plain JS) -->

  <script
    crossorigin
    src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
  ></script>
  <!-- react-dom.development.js is for DOM operations -->

  <script>
    const heading = React.createElement("h1", {}, "Hello World from React!");
    const root = ReactDOM.createRoot(document.getElementById("root"));
    root.render(heading);
  </script>
</body>
```

Q) *Why 2 CDNs - why not a single one?*

⟶ *React is used in many places: browser, mobile apps (React Native), 3D (React Three Fiber). So core React is kept separate from ReactDOM, which only talks to the browser.*

To check: open the browser console and type ***React*** *- you will see the React object. That's the CDN's work.*

Q) What if we write our script before the CDN links?

⟶ Error (React is not defined). Scripts run top to bottom, so we are using React before it has loaded.

crossorigin:

The crossorigin attribute tells the browser how to make the CORS (Cross Origin Resource Sharing) request when loading a script from a different origin (like a CDN).

```
<script crossorigin="anonymous | use-credentials">
```

anonymous ⟶ public scripts that don't need authentication (no cookies sent)

use-credentials ⟶ script needs authentication on another domain (cookies sent)

What does {} mean here?

React.createElement(tagName, **{ attributes }** , children)

Things like id, className etc. go inside the **{}** - whatever we pass there becomes the attributes of the tag.

```
<div id="root">
  <h1 id="title">Hello World!</h1>
</div>
```

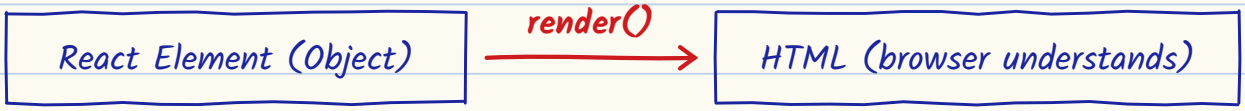
```
React.createElement("h1", { id: "title" }, "Hello World!");
```

NOTE: use className (not class) for CSS classes - class is a reserved word in JS.

NOTE: React will overwrite everything inside #root and replace it with whatever is given inside render().

React Element ⟶ a normal JS object. **props** ⟶ attributes + children we pass in.

root.render(heading) ⟶ createElement() gives a JS object, not the h1 tag. render() converts that object into a real h1 tag and puts it inside the root.



The render method does this.

DOM manipulation is one of the costliest operations in a webpage. React helps us reduce unnecessary DOM updates. In React we only write JS - we don't touch the HTML file.

Nested React elements

Q) Build this structure in React:

```
<div id="parent">
  <div id="child1">
    <h1>Heading 1</h1>
    <h2>Heading 2</h2>
  </div>
  <div id="child2">
    <h1>Heading 1</h1>
    <h2>Heading 2</h2>
  </div>
</div>
```

```
const parent = React.createElement("div", { id: "parent" }, [
  React.createElement("div", { id: "child1" }, [
    React.createElement("h1", { id: "heading_1" }, "Heading 1"),
    React.createElement("h2", { id: "heading_2" }, "Heading 2"),
  ]),
  React.createElement("div", { id: "child2" }, [
    React.createElement("h1", { id: "heading_3" }, "Heading 1"),
    React.createElement("h2", { id: "heading_4" }, "Heading 2"),
  ]),
]);

const root = ReactDOM.createRoot(document.getElementById("root"));

root.render(parent);
```

Using an Array for sibling elements.

Every id must be unique on a page - so I used heading_1 to heading_4.

(React warns about missing keys for array children - we'll learn keys later.)

*This just looks so messy and hard, so to make our life easier we have something called **JSX** .*